System Logic and Operational Workflow

The operational workflow of the Smart Railway Gate Control System is governed by an ESP32 microcontroller functioning as a sequential state machine. The system is designed to prioritize crossing safety, prevent vehicle entrapment, and autonomously alert authorities in the event of mechanical failures.

The step-by-step operational logic is defined as follows:

1. Idle State (Normal Operation)

In the absence of an approaching train, the system remains in a continuous polling state:

- **Gate Position:** The barriers remain fully open (servo motors maintained at a 90-degree position).
- **Traffic Indicators:** Green LED indicators are active, signifying safe passage for vehicular and pedestrian traffic.
- **Warning Systems:** Red LED indicators and the auditory buzzer are completely deactivated.
- **Sensor Status:** Both MFRC522 RFID readers actively poll for the presence of authorized transit tags.

2. Train Detection and Pre-Warning Protocol

The system accommodates bidirectional train traffic through dynamic reader assignment:

- **Authentication:** An approaching train passes within the proximity of an RFID reader. The microcontroller reads and verifies the unique identifier of the RFID tag.
- **Dynamic Direction Tracking:** The microcontroller registers the first triggered reader as the "Entry" node for the current cycle and designates the opposite reader as the "Exit" node.
- **Pre-Actuation Warning:** Prior to any physical movement of the barriers, the system initiates a primary warning sequence. The Green LEDs are deactivated, the Red LEDs are illuminated, and the buzzer is energized. This signals approaching road traffic to halt immediately.

3. Obstacle Detection and Safety Hold Mechanism

To mitigate the risk of vehicles becoming trapped on the tracks, the system employs an ultrasonic verification phase prior to gate closure:

- **Zone Scanning:** Following the activation of the pre-warning signals, the dual HC-SR04 ultrasonic sensors scan the crossing parameters for physical obstructions.
- **Condition A (Obstruction Detected - Safety Hold):** If a vehicle or object is detected within the crossing zone, the microcontroller initiates a "Safety Hold." Actuation of the servo motors is explicitly suspended, keeping the gates open to allow the vehicle to exit the tracks. The Red LEDs and buzzer remain continuously active to warn the driver.
- **Condition B (Clear Track):** Once the ultrasonic sensors confirm the crossing zone is completely clear of obstacles, the microcontroller transmits the PWM signal to the servo motors to lower the barriers, securing the crossing.

4. Train Departure and System Reset

- **Exit Detection:** As the train clears the crossing, its RFID tag is scanned by the designated "Exit" reader.
- **Barrier Retraction:** The microcontroller recognizes the departure and commands the servo motors to lift the barriers back to their default open position.
- **Traffic Flow Resumption:** The Red LEDs and buzzer are deactivated, and the Green LEDs are re-energized. The system clears its temporary Entry/Exit memory flags and resets to the Idle State.

5. Malfunction Detection and IoT Notification

The system features a built-in fail-safe protocol to handle hardware malfunctions, such as a physically jammed gate or a stalled servo motor:

- **Failure Identification:** If the microcontroller initiates a gate closure or opening sequence but the physical barriers fail to reach their intended positions, a malfunction flag is triggered.
- **Emergency Mode:** To prevent electrical damage or servo burnout, the system immediately halts motor actuation. It locks the traffic indicators to Red and emits a continuous emergency auditory alarm to warn approaching vehicles of a compromised crossing.
- **Automated Dispatch:** Simultaneously, the ESP32 utilizes its Wi-Fi module to interface with the Blynk IoT platform. It automatically transmits an emergency email and push notification to designated railway authorities, providing real-time notification of the specific crossing malfunction for immediate maintenance dispatch.

Exhaustive Hardware Wiring Guide

RFID-Based Smart Railway Gate Control System

System Integration Module

May 18, 2026

System Power Architecture (CRITICAL)

The ESP32 cannot supply enough current for the servos or the ultrasonic sensors. You **must** use an external 5V power supply (like a 5V 2A adapter or a buck converter).

- **Common Ground:** The Ground (GND) of the external 5V supply **MUST** be connected to the GND pin on the ESP32. If they do not share a ground, the data signals will not work.
- **3.3V Warning:** The MFRC522 RFID readers are strictly 3.3V devices. Connecting them to 5V will instantly destroy them.

1. Main Microcontroller (ESP32 DevKit V1)

ESP32 Pin	Connects To	Purpose
VIN (or 5V)	External 5V (+)	Powers the ESP32 from the external supply
GND	External Supply (-)	Establishes the Common Ground
3V3	RFID Readers (3.3V)	Provides safe 3.3V power to both RFID readers

2. RFID Readers (MFRC522) - 3.3V Logic

Component Pin	Reader 1 (Entry Gate)	Reader 2 (Exit Gate)	Wire Type
3.3V	ESP32 **3V3** Pin	ESP32 **3V3** Pin	Power
RST	ESP32 **GPIO 22**	ESP32 **GPIO 21**	Data
GND	Common Ground	Common Ground	Ground
IRQ	*Not Connected (NC)*	*Not Connected (NC)*	None
MISO	ESP32 **GPIO 19**	ESP32 **GPIO 19**	Data (Shared)
MOSI	ESP32 **GPIO 23**	ESP32 **GPIO 23**	Data (Shared)
SCK	ESP32 **GPIO 18**	ESP32 **GPIO 18**	Data (Shared)
SDA / SS	ESP32 **GPIO 5**	ESP32 **GPIO 4**	Data (Unique)

3. Ultrasonic Obstacle Sensors (HC-SR04) - 5V Logic

Component Pin	Sensor 1 (Road A)	Sensor 2 (Road B)	Wire Type
VCC	External **5V** Supply	External **5V** Supply	Power
Trig	ESP32 **GPIO 13**	ESP32 **GPIO 25**	Data (Output)
Echo	ESP32 **GPIO 14**	ESP32 **GPIO 26**	Data (Input)
GND	Common Ground	Common Ground	Ground

4. Gate Actuators (Servo Motors) - 5V Power

Component Pin	Servo 1 (Entry Gate)	Servo 2 (Exit Gate)	Wire Type
VCC (Red)	External **5V** Supply	External **5V** Supply	Power
GND (Brown/Blk)	Common Ground	Common Ground	Ground
Signal (Orange/Yel)	ESP32 **GPIO 27**	ESP32 **GPIO 32**	PWM Data

5. Traffic Lights & Buzzer

For the LEDs, the long leg is the Anode (+), and the short leg is the Cathode (-). You **must** put a 220Ω or 330Ω resistor between the ESP32 pin and the LED to prevent burning out the LED or the ESP32 pin.

Component	Pin / Leg	Connection Destination
Red LED 1 (Entry)	Anode (+) Cathode (-)	To 220Ω Resistor, then to **GPIO 33** Common Ground
Green LED 1 (Entry)	Anode (+) Cathode (-)	To 220Ω Resistor, then to **GPIO 16** Common Ground
Red LED 2 (Exit)	Anode (+) Cathode (-)	To 220Ω Resistor, then to **GPIO 33** Common Ground
Green LED 2 (Exit)	Anode (+) Cathode (-)	To 220Ω Resistor, then to **GPIO 16** Common Ground
Buzzer (Active)	Positive (+) Negative (-)	**GPIO 17** Common Ground

Member 1 (REZWAN): RFID Train Detection & Authentication

- **Primary Responsibility:** Managing the primary trigger mechanism for the entire state machine.
- **Hardware Focus:** Wiring and calibrating the two MFRC522 RFID readers over the shared SPI bus (Pins 18, 19, 23).
- **Software Focus:** Writing the checkRFID() function. This includes reading the UID of the tags, verifying them against the authorized array, and managing the bidirectional logic to determine which reader acts as the "Entry" and which acts as the "Exit" for a given cycle.

Member 2 (SIFAT): Servo Gate Control & Co-Lead Integrator

- **Primary Responsibility:** Physical actuation of the barriers and maintaining the master code repository.
- **Hardware Focus:** Connecting the MG995/MG996R servo motors (Pins 27, 32) and ensuring they receive safe power from the external 5V supply rather than the ESP32 to prevent brownouts.
- **Software Focus:** Implementing the ESP32Servo.h library, writing the closeGates() and setIdleState() functions, and dialing in the exact PWM angles for fully open and fully closed states.
- **Integration Duty:** Working directly with P3 to merge all individual C++ functions into the main loop() switch-case state machine.

Member 3 (UDOY): Ultrasonic Obstacle Detection & Co-Lead Integrator

- **Primary Responsibility:** Implementing the critical fail-safe logic to prevent vehicle entrapment.
- **Hardware Focus:** Installing and wiring the two HC-SR04 sonar sensors (Pins 13, 14, 25, 26).
- **Software Focus:** Writing the getDistance() and isCrossingClear() functions. This requires handling the pulseIn() timing logic with proper timeouts (to prevent the ESP32 from freezing) and managing the SAFETY_HOLD_CHECK state.
- **Integration Duty:** Working directly with P2 to merge the safety-hold logic with the servo actuation, ensuring the gates physically cannot close if an obstacle is present.

Member 4 (Saccha): IoT Architecture & Cloud Monitoring

- **Primary Responsibility:** Bringing the system online and handling emergency communications.
- **Hardware Focus:** Ensuring the ESP32 maintains a stable Wi-Fi connection without interrupting the hardware timers.
- **Software Focus:** Setting up the BlynkSimpleEsp32.h integration. This includes configuring the Blynk Web Dashboard, managing the Wi-Fi credentials, writing the triggerMalfunction() timeout logic (the 10-second rule), and pushing the emergency email alerts.

Member 5 (ESHA): Warning Systems & Power Distribution

- **Primary Responsibility:** Managing the physical alerts and ensuring the overall circuit is wired cleanly and safely.
- **Hardware Focus:** Wiring the Red LEDs, Green LEDs, and the active Buzzer (Pins 16, 33, 17) with their respective 220Ω resistors. Assisting the team by managing the shared Common Ground on the breadboard.
- **Software Focus:** Writing the triggerPreWarning() function. This member is responsible for the transition state that flashes the red lights and sounds the alarm *before* the servos are allowed to move.